

Shortest Path Algorithms

Optimizing Connectivity and Path Weight Costs

1. The Shortest Path Problem

The Shortest Path Problem is a cornerstone of graph theory, aiming to find a route between two vertices such that the sum of the weights of its constituent edges is minimized. This is applied daily in GPS navigation, network packet routing, and logistics.

Path Cost: Also known as path weight, this is the total sum of all edge weights along a specific trajectory from point A to point B.

2. Edge Relaxation

Most shortest-path algorithms rely on a technique called **Relaxation**. To "relax" an edge (u, v) means to check if the current known distance to node v can be improved by going through node u .

```
if distance[u] + weight(u, v) < distance[v]:  
    distance[v] = distance[u] + weight(u, v)
```

3. Dijkstra vs. Bellman-Ford

Depending on the nature of the edge weights, different algorithms are required:

Algorithm	Edge Weights	Performance
Dijkstra	Positive only	$O(E \log V)$ - Fast
Bellman-Ford	Positive & Negative	$O(VE)$ - Slower

4. The Problem with Negative Cycles

A **Negative Cycle** occurs when the sum of edge weights in a loop is less than zero. In such a graph, a "shortest path" does not exist because you could loop infinitely to reach a distance of -Infinity.

Fact: Dijkstra's algorithm will fail or provide incorrect results in the presence of any negative edge. Bellman-Ford can handle negative edges and effectively *detect* negative cycles.

5. Real-World Analogy

Imagine locations as vertices and roads as edges:

- **Positive Weight:** Distance in kilometers or fuel cost.
- **Negative Weight:** A road where you perform a service (like delivery) and earn more money than the cost of the travel.

Mathematical Foundations (Part 1)

The concept of **Infinity Initialization** is vital. Before the algorithm starts, we assume all distances are ∞ except for the source node, which is 0. This sets the stage for the relaxation process to progressively "lower" the cost to each node.

Triangle Inequality: In a stable shortest path tree, for any edge (u, v) , the distance to v must be less than or equal to the distance to u plus the weight of the edge between them. If this isn't true, the edge can still be relaxed.

Why Dijkstra is Greedy

Dijkstra is considered a **Greedy Algorithm** because at every step it selects the "closest" unvisited vertex, assuming that the shortest path found to that node is final. This is why it fails with negative weights—a negative edge discovered later could have made a previous "longer" path actually shorter.

Mathematical Foundations (Part 2)

The concept of **Infinity Initialization** is vital. Before the algorithm starts, we assume all distances are ∞ except for the source node, which is 0. This sets the stage for the relaxation process to progressively "lower" the cost to each node.

Triangle Inequality: In a stable shortest path tree, for any edge (u, v) , the distance to v must be less than or equal to the distance to u plus the weight of the edge between them. If this isn't true, the edge can still be relaxed.

Why Dijkstra is Greedy

Dijkstra is considered a **Greedy Algorithm** because at every step it selects the "closest" unvisited vertex, assuming that the shortest path found to that node is final. This is why it fails with negative weights—a negative edge discovered later could have made a previous "longer" path actually shorter.

Mathematical Foundations (Part 3)

The concept of **Infinity Initialization** is vital. Before the algorithm starts, we assume all distances are ∞ except for the source node, which is 0. This sets the stage for the relaxation process to progressively "lower" the cost to each node.

Triangle Inequality: In a stable shortest path tree, for any edge (u, v) , the distance to v must be less than or equal to the distance to u plus the weight of the edge between them. If this isn't true, the edge can still be relaxed.

Why Dijkstra is Greedy

Dijkstra is considered a **Greedy Algorithm** because at every step it selects the "closest" unvisited vertex, assuming that the shortest path found to that node is final. This is why it fails with negative weights—a negative edge discovered later could have made a previous "longer" path actually shorter.